

USAUSA TRAINING WORKSHOP

AI駆動開発のテスト設計

商用プロジェクトで気をつけること──新人向け／中堅向け

査読済み論文ベース

新人向け

過信しない・基礎力とのバランス

中堅向け

レビュー負荷・生産性実感のズレ

商用現場

検証プロセスの具体的な見直し

OVERVIEW

なぜ今、AI駆動開発のテスト設計が重要か

AIによるコード生成は開発速度を押し上げる一方、検証プロセスを飛ばすと商用プロジェクトで事故につながります。3つの観点から整理します。

1	**生産性への効果**	特定のタスクで作業完了が速くなったとする報告がある（査読前を含む）	Peng et al.(2023)、Cui et al.(2024)
2	**検証課題**	AI生成コードには、気づきにくい欠陥・脆弱性が混入しやすい	Perry et al.(2023)、Vaithilingam et al.(2022)、Dakhel et al.(2023)
3	**経験者ほどのワナ**	「速くなった感覚」と実際の生産性にズレが生じることがある	Becker et al. / METR(2025)

RESEARCH 1

AIによる生産性への効果

Peng, Kalliamvakou, Cihon & Demirer (2023)[プレプリント・査読前]

GitHub Copilotを使ったランダム化比較試験（RCT）で、特定のコーディングタスクにおいて開発者の作業完了速度が有意に向上したことを報告。AIの効果を定量的に検証した代表的な研究の1つ。

Cui et al. (2024)[ワーキングペーパー・査読前]

企業内の大規模な開発チームを対象に、AIコーディング支援ツール導入前後の生産性を比較。特に経験の浅い開発者で効果が大きい傾向が報告されている。

RESEARCH 2

AI生成コードの検証課題

Perry, Srivastava, Kumar & Boneh (2023)

AI支援を受けた開発者は、受けなかった開発者よりもセキュリティ上の脆弱性を含むコードを書く傾向があり、しかも自分のコードを「安全だ」と過信する傾向も確認された。

Vaithilingam, Zhang & Glassman (2022)

AIが生成したコードを、開発者が正しく検証できているとは限らないことを、ユーザビリティ評価の実験で示した。生成結果への理解不足が検証漏れにつながる。

Dakhel et al. (2023)

GitHub Copilotが生成するコードには、正確性の問題（期待通りに動かないケース）が一定数含まれ、人間によるレビューが欠かせないと報告。

新人向け

気をつけること

背景

Cui et al.(2024)（査読前）では経験の浅い開発者ほど効果が大きい傾向が報告されています。基礎力が育つ前にAI任せにすると検証できなくなる懸念は本資料の提案です（Perry et al. 2023の過信リスクが示唆）。

チェックリスト

- 生成されたコードを「読んで理解できる」まで、提出・コミットしない
- なぜそのコードで動くのか、自分の言葉で説明できるか確認する（Perry et al. 2023の過信リスクを避けるため）
- 基礎文法・アルゴリズムの学習は、AIに頼らず手を動かす時間を意図的に確保する（Sweller 1988：基礎が身についていないと、応用の認知負荷に耐えられない）
- わからないまま採用した処理は、必ず先輩にレビューしてもらう

中堅向け

気をつけること

背景

Becker et al. / METR(2025)（プレプリント・査読前）の研究では、経験豊富な開発者がAIツールを使うと「速くなった」と感じる一方、実際の作業時間を計測すると遅くなっているケースが報告されました。主観的な実感と実際の生産性にズレが生じうることを示しています。

チェックリスト

- 「速く感じる」ことと「実際に速い」ことを分けて考え、定期的に計測する
- レビュー負荷の増加を見込んで、AI生成コード向けのレビュー観点（セキュリティ・例外処理・暗黙の前提）を明文化しておく
- 自分の暗黙知（これまで無意識にチェックしていたこと）がAIの出力には反映されない前提で、チェックリスト化しておく
- 新人のAI活用を監督する際は、本人が内容を説明できるかを確認する

PRACTICE

商用プロジェクトのテスト設計で実践すること

1	**セキュリティテストの強化**	AI生成コードは脆弱性混入率が上がりうるため、静的解析・セキュリティレビューを通常より厳格に設定する
2	**カバレッジ基準の見直し**	「動いた」ことと「正しい」ことは別。境界値・異常系のテストケースを人間側で追加する
3	**レビュー観点の明文化**	暗黙知に頼らず、AI生成コード特有のチェック項目（例外処理・前提条件）をリスト化し、属人化を防ぐ
4	**計測の習慣化**	「速くなった感覚」に頼らず、リードタイムや手戻り率を定期的に計測する

SUMMARY

新人向け／中堅向けの違い

新人向け

- 主なリスク：わからないまま採用してしまう「過信」
- 対策の軸：読んで説明できる状態まで検証する
- 並行して基礎力（AIに頼らない作業時間）を確保する

中堅向け

- 主なリスク：「速くなった感覚」と実際の生産性のズレ
- 対策の軸：暗黙知をチェックリスト化し、定期的に計測する
- 新人のAI活用を、説明できるかどうかで監督する

REFERENCES

参考文献

・Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The Impact of AI on Developer Productivity: Evidence from GitHub Copilot.（プレプリント）

・Cui, Z., et al. (2024). The Effects of Generative AI on High-Skilled Work: Evidence from Software Development.（ワーキングペーパー）

・Becker, J., et al. / METR (2025). Measuring the Impact of AI on Experienced Open-Source Developer Productivity.（プレプリント）

・Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do Users Write More Insecure Code with AI Assistants? ACM CCS.

・Vaithilingam, P., Zhang, T., & Glassman, E. L. (2022). Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by LLMs. CHI EA.

・Dakhel, A. M., et al. (2023). GitHub Copilot AI pair programmer: Asset or liability? Journal of Systems and Software.

・Sweller, J. (1988). Cognitive load during problem solving. Cognitive Science.



検証を、AI任せにしない

うさうさ研修工房 ― 面白きこともなき世を面白く